

Git

Configurando o GitHub

Configurações iniciais de identidade do usuário.

- `git config --global --list`: Retorna as configurações de usuário atuais.
- `git config --global user.name "nome"`: Define o nome do usuário da máquina.
- `git config --global user.email "email"`: Define o email do usuário da máquina.

Começo de um Repositório

Iniciando e monitorando o estado do projeto.

- `git init`: Inicia um repositório vazio no diretório atual.
- `git status`: Devolve o status atual do repositório (qual branch está, arquivos rastreados e não rastreados).
- `git log`: Mostra o histórico de commits detalhadamente.
- `git log --oneline`: Mostra o histórico de commits de forma simplificada (uma linha por commit).

Commits

Gerenciamento de alterações e histórico.

- `git add <arquivo>`: Adiciona o arquivo ou diretório aos arquivos rastreados (Staging Area).
 - **Dica:** Use `.` no lugar do nome do arquivo para adicionar **tudo** no diretório atual.
- `git rm --cached <arquivo>`: Retira o arquivo da área de rastreo (Unstage), mas mantém o arquivo no disco.
- `git commit -m "mensagem"`: Grava os arquivos referenciados no histórico da branch atual.

- `git checkout <id>`: Retorna para um commit anterior sem alterar o histórico (modo “Detached HEAD”).
- `git checkout master`: Retorna para o commit mais recente da branch master (ou main).
- `git revert <id>`: Cria um **novo** commit revertendo as alterações de um commit anterior. É seguro pois não apaga o histórico.
- `git reset <id>`: Retorna para um commit anterior e apaga os commits subsequentes, mas **mantém** as alterações nos arquivos (considera como não rastreadas/modificadas).
- `git reset --hard <id>`: Retorna para um commit anterior e limpa tudo (apaga alterações nos arquivos). Cuidado, `git reset` altera o histórico. A versão `--hard` não tem volta.
 - **Nota:** Use `:q` para sair da tela de visualização/aviso no terminal.

Ignorando Arquivos: Crie um arquivo chamado `.gitignore` e escreva o nome de diretórios ou arquivos que não devem ser rastreados pelo Git.

Branches (Ramificações)

Trabalhando em paralelo.

- `git branch`: Mostra todas as branches disponíveis.
- `git branch <nome>`: Cria uma branch nova baseada na atual.
- `git checkout <nome>`: Move para o commit mais recente da branch especificada.
- `git branch -d <nome>`: Deleta a branch especificada.
 - Use `-D` (maiúsculo) para forçar a deleção ignorando segurança de arquivos não mergeados.

Merge

Integrando ramificações à branch atual (ex: master).

- `git merge <nome>`: Cria novos commits na branch atual trazendo o histórico da branch <nome>.
 - **Conflitos**: Se houver conflito, é necessário resolver manualmente no código e fazer um novo commit para finalizar.

Repositórios Remotos

Interagindo com a nuvem (GitHub/GitLab).

- `git clone <url>`: Baixa um repositório remoto completo para sua máquina.
- `git remote add origin <url>`: Conecta seu repositório local atual a um repositório vazio na nuvem.
- `git remote -v`: Lista os repositórios remotos vinculados.
- `git push`: Envia seus commits locais para o repositório remoto.
 - **Primeira vez**: Use `git push --set-upstream origin main` para ligar as branches.
- `git pull`: Baixa atualizações do remoto e faz o merge automaticamente com sua branch atual.
- `git fetch`: Baixa as atualizações do remoto mas **não** aplica (seguro para ver o que mudou antes de integrar).

Stashing

Guarda mudanças “suja” para limpar o diretório de trabalho sem precisar commitar. Útil para trocas rápidas de contexto.

- `git stash`: Pega arquivos modificados (não commitados) e os salva numa pilha temporária, limpando o diretório.
- `git stash pop`: Devolve as últimas mudanças salvas para o diretório e as remove da pilha.
- `git stash list`: Mostra a lista de mudanças guardadas na pilha.

Tags

Marcando pontos específicos na história como lançamentos oficiais (ex: v1.0).

- `git tag`: Lista todas as tags existentes.
- `git tag -a <v1.0> -m "mensagem"`: Cria uma tag anotada (recomendado para releases).
- `git push origin <tag>`: Envia a tag específica para o repositório remoto (tags não sobem no `git push` normal).

Worktrees

Permite gerenciar múltiplos diretórios de trabalho ligados ao mesmo repositório. Útil para trabalhar em duas branches diferentes simultaneamente sem precisar fazer **stash** ou clonar o repo novamente.

- `git worktree list`: Lista todos os worktrees ativos e seus diretórios.
- `git worktree add <caminho> <branch>`: Cria um novo diretório (caminho) contendo o checkout da branch especificada.
- `git worktree remove <caminho>`: Remove o worktree e desvincula a pasta do repositório.
- `git worktree prune`: Limpa informações de worktrees que já foram deletados manualmente do disco.